# Jigsaw Image Processing – Milestone 1 Technical Report

## Faculty of Engineering – Ain Shams University

## CSE483 / CESS5004

*Submitted To:*
*Dr. Mahmoud Khalil*
*TA/ Dina Zakaria*
*TA/ Ali Ahmed*

**Team Members:**

1. Omar Ahmed Abouraia 23P0100
2. Hossam Ossama Hussieny 23P0010
3. Mariam Ibrahim Assr 23P0123
4. Mariam Tarek Farouk 23P0214
5. Logaine Mohamed 23P0187

- **GitHub Repo Link**

## Table of Contents

1. Introduction ..... 3
2. Methodology ..... 3
   2.1 Preprocessing ..... 3
   2.2 Segmentation ..... 3
   2.3 Feature Extraction ..... 3
3. Implementation Details ..... 3
4. Techniques Attempted, Rationale, Failure Cases, and Suitability for Assembly ..... 4
5. Conclusion ..... 4
6. Visual Pipeline Results ..... 5
   6.1 Original 8×8 Puzzle Image ..... 5
   6.2 After Applying Preprocessing & Edge Operations ..... 6

# 1. Introduction

This report summarizes Milestone 1 of the Jigsaw Image Processing project, which focuses on transforming raw puzzle images into structured, enhanced, and segmented puzzle tiles. The milestone implements robust preprocessing, grid segmentation, and contour-based feature extraction. These steps form the foundation for Milestone 2, where puzzle-piece matching and assembly will occur.

# 2. Methodology

## 2.1 Preprocessing

The preprocessing pipeline includes:
- Gaussian Blur: Reduces high-frequency image noise.
- Grayscale Conversion: Simplifies processing by reducing to one channel.
- Median Blur: Suppresses salt-and-pepper noise.
- CLAHE: Enhances local contrast and highlights edges.
- Sharpening Filter: Enhances boundaries for contour extraction.
- Gamma Correction: Normalizes lighting and contrast across tiles.

## 2.2 Segmentation

The puzzle grid size is detected automatically from the filename or directory structure, supporting 2x2, 4x4, and 8x8 puzzles. The image is divided into equal tiles using array slicing. Each tile is saved and processed independently to extract contours and features.

## 2.3 Feature Extraction

For each tile, Canny edge detection identifies prominent boundaries, and OpenCV's contour extraction locates puzzle-edge outlines. Features computed include:
- Contour Area
- Perimeter
- Bounding Box
- Centroid
- Hu Moments (rotation-invariant)
- Number of Polygon Vertices

These descriptors support robust edge comparison in later puzzle assembly.

# 3. Implementation Details

The project uses Python, OpenCV, NumPy, and JSON for feature storage. The output is organized into:
- tiles/: preprocessed puzzle tiles
- edges/: Canny edge maps
- contours/: contour overlays
- visualizations/: grid visualization and full-image contour map

- contours.json: structured feature dataset
This organization ensures clarity, modularity, and ease of debugging.

## 4. Techniques Attempted, Rationale, Failure Cases, and Suitability for Assembly

Throughout the development of Milestone 1, several classical image-processing techniques were explored with the objective of enhancing puzzle boundaries and creating reliable inputs for later edge-matching.

Techniques Attempted:
- Gaussian Blur, Median Blur, and Bilateral Filtering were tested. Gaussian blur reduced high-frequency noise, while median blur removed salt-and-pepper noise effectively.
- Global histogram equalization was attempted but resulted in excessive contrast. CLAHE was selected instead for its controlled enhancement of local contrast.
- Multiple sharpening kernels were tested, with the unsharp-mask style kernel achieving the best clarity.
- Gamma correction was tuned carefully—too low made the image too dark; the chosen value balanced contrast.

Failure Cases:
- Low-contrast tiles sometimes failed to produce strong contours.
- Highly textured regions produced false-positive edges.
- Early iterations using aggressive sharpening produced amplified noise.

Suitability for Later Assembly:
The outputs—preprocessed tiles, edge maps, contours, and extracted Hu moments—form a structured dataset essential for puzzle assembly. Clean contours allow reliable geometric comparison, while rotation-invariant Hu Moments enable matching even when pieces appear rotated. The JSON feature dataset ensures compatibility with automated matching in Milestone 2.

## 5. Conclusion

Milestone 1 successfully produces enhanced images, structured tile segmentation, and a complete set of shape descriptors for puzzle pieces. These artifacts prepare the groundwork for Milestone 2, which focuses on identifying matching edges and assembling puzzle fragments.

## 6. Visual Pipeline Results

### 6.1 Original 8×8 Puzzle Image

**Figure 1 : Original 8×8 puzzle image before preprocessing.**

## 6.2 After Applying Preprocessing & Edge Operations

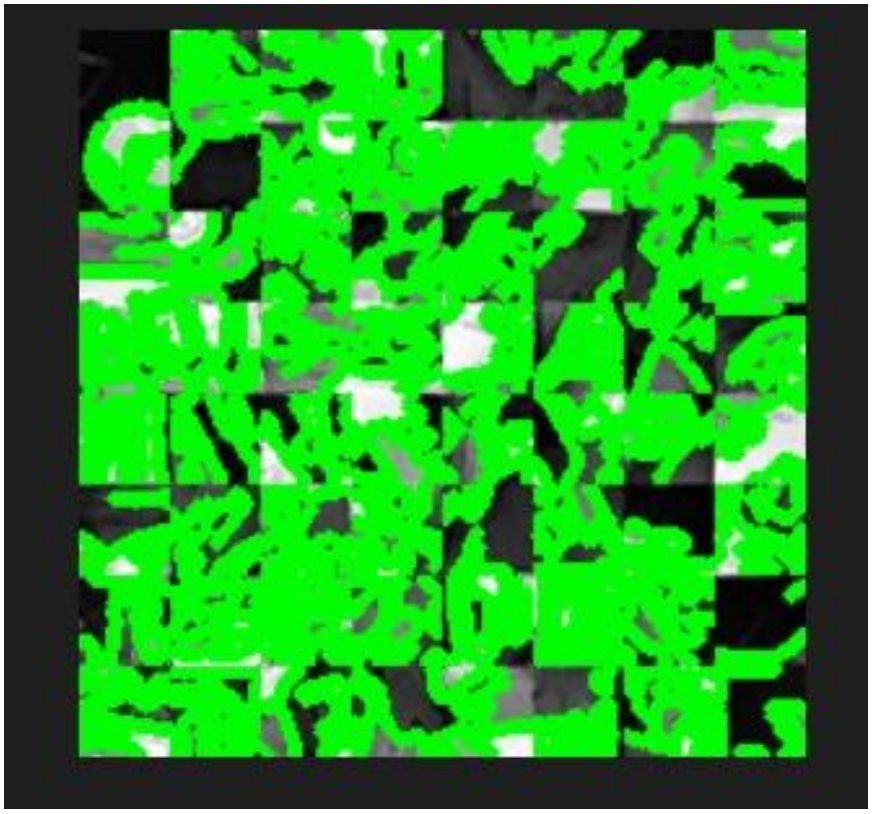

**Figure 2: Edge map and contour overlay after preprocessing.**

This intermediate stage shows the binary/edge-enhanced output, highlighting detected contours across all tiles. The green overlay represents extracted boundaries used for feature computation.